Lab 7 — Turning Your Workflow into Team Tooling

After Module 7 — Copilot Customization

In the last six labs you typed the same kinds of prompts over and over. Now you will encode them, so the whole team gets the same result without remembering the prompt.

Before you start: the starter repository already contains .github/prompts/pr-prep.prompt.md, .github/agents/planner.agent.md, and two example test files for the skill you are about to build. You will use all of them.

Start on a clean main with Lab 6 merged.

```
git checkout -b customization
```

Part 1 — Prompt Files

Step 1 — Write a draft prompt file

Every prompt file starts as a draft in Ask mode. It cannot damage anything while you are still iterating on it.

Create .github/prompts/draft-new-endpoint.prompt.md:

```
---
agent: 'ask'
model: 'Claude Sonnet 5'
description: 'Add a new REST endpoint following project conventions'
argument-hint: 'HTTP method, path, and what it should do'
---

Add a new endpoint to the TodoApp backend.

Requested endpoint: ${input:endpoint}

Follow the layer rules in .github/copilot-instructions.md and
.github/instructions/backend.instructions.md exactly:
 - The router handles HTTP only. It calls a service. It never imports
   the repository.
 - The service holds business rules. It never imports FastAPI.
 - The repository is the only code that touches the database.

For this endpoint produce, in this order:
 1. The Pydantic request and response schemas, if new ones are needed.
```

```
2. The repository method, with the SQLAlchemy query. Filtering and
   sorting happen in SQL, never in Python.
3. The service method, including validation and the error case.
4. The router function, with the correct status code:
   201 create, 200 read, 204 delete, 404 unknown id, 422 invalid input.
5. A pytest test per outcome, including the failure path.
6. A one-line note stating whether an Alembic migration is required.

If the request is ambiguous about the response shape or the error
behaviour, ask before producing anything.
```

Step 2 — Test it

Open the prompt file in the editor. Click the Play button at the top right. It runs in a scratch chat. When prompted for the argument, enter:

```
GET /todos/overdue - todos created more than 7 days ago that are not completed
```

Read the output, then check three things:

- Expand the References list. Confirm copilot-instructions.md and backend.instructions.md were actually loaded. If they were not, the prompt file is relying on conventions Copilot cannot see.
- Did it ask a clarifying question, or did it invent the response shape? A good prompt file asks.
- Did it put the query in the repository, or did it filter in Python?

Step 3 — Iterate

Fix whatever was wrong. Run it again with a different argument:

```
DELETE /todos/stale - delete every todo older than 90 days, return the count
```

If the output varies wildly between the two runs in structure, not just content, your prompt is under-specified. Tighten it.

Step 4 — Promote it

Only when you trust it:

- Change agent: 'ask' to agent: 'agent'.
- Add a tools line to the frontmatter:

```
tools: ['search/codebase', 'search/usages', 'editFiles']
```

- Rename the file from draft-new-endpoint.prompt.md to new-endpoint.prompt.md.

In Copilot Chat, type / and look at the list. new-endpoint now appears with your description next to it. Run it:

```
/new-endpoint GET /todos/overdue - todos created more than 7 days ago that are not completed
```

It writes the files. Run the tests.

Part 2 — Agent Skills

A prompt file is a prompt you invoke. A skill is a capability Copilot loads by itself when the task matches.

Step 6 — Write SKILL.md

Create the folder .github/skills/todoapp-testing/ and inside it create SKILL.md:

```
---
name: todoapp-testing
description: How to write tests for the TodoApp project. Use whenever
  asked to write, fix, extend, or review tests for the FastAPI backend
  or the React frontend of this project, including pytest tests, Vitest
  tests, fixtures, mocks, and coverage work.
---

# Writing tests for TodoApp

This skill defines how tests are written in this project. Follow it for
every test you generate.

## When to use this skill
Any request involving tests: writing new tests, fixing failing tests,
closing a coverage gap, adding a regression test for a bug, or reviewing
an existing test file.

## Backend tests

Location: `backend/tests/`. Framework: pytest.

- Service tests mock the repository. They never touch a database.
- Router tests use the TestClient fixture from `conftest.py`, which
```

binds a fresh temporary database per test. Never use `todos.db`.
- Name every test `test_<method>_<condition>_<expected>`.
- Use Arrange-Act-Assert with a blank line between the three sections.
- Use `@pytest.mark.parametrize` for boundary tables. Do not write five near-identical test functions.
- Assert the exact status code, never just `response.ok`.
- Every bug fix gets a regression test named after the bug, with a comment linking the issue number.

Frontend tests

Location: `frontend/src/__tests__/`. Framework: Vitest with React Testing Library.

- Always mock `src/api/todoApi.js`. Tests never make network calls.
- Query by accessible role or label, not by CSS class or test id.
- Assert on what a user would see, not on component internal state.

Required edge cases

Every test suite that touches a todo must cover:
empty title, title at exactly 200 characters, title at 201 characters, whitespace-only title, category of None, category of empty string, and an id that does not exist.

Never do

- Never delete or weaken a failing test to make a suite green.
- Never assert on a datetime with an exact equality check. Use a range.
- Never write a test that passes when the production code is deleted.

Reference files

See `example-service-test.py` and `example-api-test.py` in this folder for the exact style expected.

Copy the two example files from the starter repository into the same folder. They are loaded only when the skill body references them, so they cost nothing until they are needed.

Step 7 — Trigger the skill without naming it

Open a fresh chat. Do not mention the skill. Ask:

```
Write tests for the overdue endpoint.
```

Expand the References list. SKILL.md should appear — Copilot matched your request against the skill's description field and loaded it. If it did not load, the description is the problem, not the skill body.

Now read the generated tests. Confirm they use the naming convention, parametrize the boundaries, and mock the repository — none of which you asked for in the prompt. That is the difference between a skill and a prompt file. You never invoked it.

Part 3 — A custom agent with a handoff chain

You already have a Planner and a Refactorer. Now add the third role and wire the chain.

Step 8 — Create the Reviewer agent

Create .github/agents/reviewer.agent.md:

```
---
name: Reviewer
description: Reviews a diff for correctness and layer violations
tools: ['search/codebase', 'search/usages', 'search/changes', 'read/problems']
model: ['Claude Sonnet 5', 'GPT-5.6 Terra']
handoffs:
  - label: Apply Fixes
    agent: agent
    send: false
  - label: Prepare Pull Request
    agent: ask
    prompt: 'Run the pr-prep prompt file against this branch.'
    send: false
---

# Instructions
```

```
You are the Reviewer for TodoApp. You have no ability to edit files,
and that is deliberate. You find problems. Someone else fixes them.

Review the changes in this branch using #changes.

Apply the rules in .github/instructions/code-review.instructions.md.

For every issue report exactly four things:
  1. File and line.
  2. The rule it breaks.
  3. Severity: blocking, should-fix, or nit.
  4. The smallest fix, described in one sentence. Not the code.

Group your output by severity, blocking first. If there are no blocking
issues, say so in the first line.

Then state the single riskiest change in the diff and how a human would
verify it.

Never comment on formatting, line length, or missing type hints on
private helpers.
```

Step 9 — Run the chain

Make a small change by hand: add a GET /todos/count endpoint, and deliberately put the SQLAlchemy query in the router instead of the repository.

Now run the agents in sequence:

- Mode picker, select Reviewer. Ask: Review the changes on this branch.
- Read the report. It should flag the layer violation as blocking.
- Click Apply Fixes. Read the pre-filled prompt before you send it — send: false exists so you can.
- Send it. The default agent applies the fix.
- Return to Reviewer and review again. Confirm the blocking issue is gone.

Step 10 — Ship it

```
git add .
```

```
git commit -m "Add prompt file, testing skill and reviewer agent"
git push -u origin customization
```

Open a pull request. Use the supplied /pr-prep prompt file to write the description. Request a Copilot review. Merge.

What you should have at the end of this lab

```
.github/
 prompts/
  new-endpoint.prompt.md
  pr-prep.prompt.md       (supplied)
 skills/
  todoapp-testing/
   SKILL.md
   example-service-test.py  (supplied)
   example-api-test.py     (supplied)
 agents/
  planner.agent.md       (supplied)
  refactorer.agent.md
  reviewer.agent.md
```

Everything you did by hand in Labs 1 to 6 is now something a new team member gets for free on their first clone.